

Quantum Terminal Pro

Technical Handover Manual: Architecture, Deployment, and Operations

1. Platform Architecture Overview

Quantum Terminal Pro is an enterprise-grade, high-performance SaaS platform designed for the real-time ingestion, aggregation, and visualization of low-latency market data. The platform focuses primarily on cryptocurrency derivatives (perpetual contracts) across top-tier exchanges, providing deep telemetry data such as liquidation heatmaps, volume profiles, Cumulative Volume Delta (CVD), and order book imbalance metrics.

The system utilizes an asynchronous, event-driven microservices architecture built around **Redis** as a central high-speed message broker and in-memory data store. All core worker processes are implemented using Python's native `asyncio` framework.

Core System Components:

- **Ingestion Modules (Ingesters):** 9 independent WebSocket clients (Binance, Bybit, OKX, Bitget, Deribit, Hyperliquid, dYdX, HTX, Gate). These services maintain persistent connections with exchanges, normalise incoming ticks, monitor large whale trades, track order book changes, and capture forced liquidation events, streaming them directly into a Redis Stream.
- **Minute Aggregator (Aggregator / Minute Builder):** A consumer microservice that reads raw tick data from the Redis Stream using Consumer Groups. It aggregates market metrics every minute, building 1-minute candlestick snapshots and persisting them into Redis Hash structures with an optimized TTL of 48 hours.
- **Trend Calculator (Comparator / Trend Calculator):** A module that computes high-order moving averages (EMA 50, EMA 200, EMA 800), global Open Interest (OI), and cumulative volume metrics. It saves sliding historical snapshots into Redis Lists for rapid client-side delivery.
- **API Server:** A high-throughput web server powered by FastAPI and Uvicorn. It handles real-time user authentication, manages secure client WebSocket broadcasts, and provides specialized REST endpoints, including dynamic CSV stream generation tailored for MetaTrader 5 (MT5) terminals.
- **Data Archiver (Archiver / Daily Snapshot):** A automated daily backup cron-like service that fetches all aggregated 1-minute historical data from Redis for the previous day and writes them into highly compressed, query-optimized Apache Parquet files for long-term cold storage.
- **Visualization Dashboard:** A client-facing front-end built using the TradingView Lightweight Charts library. It streams real-time prices, large-scale order tapes, order book imbalance indicators, and custom analytical data overlays.

2. Infrastructure & Redis Data Schema

Redis serves as both the low-latency transport layer and the real-time operational data cache. Below is the technical specification of the memory schemas and key design patterns used:

Data Type	Key Pattern	Description & Operational Purpose
Stream	raw_market_stream	Central high-throughput ingestion pipeline for raw market ticks. Utilizes Consumer Groups to safely distribute the load across workers.
Hash	live_metrics:{Symbol}	Real-time streaming metrics (current mark price, order book bid/ask volumes per exchange, open interest). Updated every second by ingesters.
Hash	agg:minute:{Symbol}:{Timestamp}	Aggregated per-minute metrics (buy/sell volume, liquidation intensity per exchange). Configured with a strict 48-hour TTL.
Hash	liq:heatmap:history:{Symbol}:{HourSlot}	Historical order book liquidation density grids used to render client-side Heatmaps. Configured with a 24-hour TTL.
List	chart:history:{Symbol}	Maintains the last 1,440 data points (24 hours) of pre-calculated technical indicators (EMAs, CVD, global OI) for instant chart initialization upon user connection.
Hash	valid_api_keys	A global registry containing authorized commercial API keys. Field maps the token to the client's corporate name.

3. Security, Licensing & Access Control

The platform implements a hybrid security protocol designed to handle both standard web client authentications and secure programmatic access from backend algorithmic trading scripts (MT5).

License Categories & Validation Rules:

- **Demo Web Keys (DEMO-***):** Temporary credentials intended for evaluation. Upon its first usage, the key is removed from the demo_keys_pending queue and activated inside Redis under demo_active:{key} for exactly 1 hour (3,600 seconds).
- **MT5 Licenses (MT5DEMO / MT5_SECURE):** Dedicated endpoints utilized by Expert Advisors. To prevent credential sharing or cloning, the system tracks the client's source IP address. For demo keys, once the 1-hour expiration is met, the system enforces a strict 30-day (2,592,000 seconds) IP block in Redis under demo_ip_used:{client_ip}.

CRITICAL SECURITY VULNERABILITY ALERT: In the current implementation of api_server/mt5_auth.py, the hashing salt is hardcoded directly into the source file: SECRET_SALT = "QuantumTerminalPro_SuperSecret_2026!". Before completing the final production handover, **this variable must be moved** to an environment variable. The salt is vital because it creates the verification SHA-256 hash for accounts: hashlib.sha256(f"{mt5_account}{SECRET_SALT}".encode()).hexdigest(). Changing this value requires coordinating the update with the trading robots' authentication headers.

4. Environment Configuration (.env)

All infrastructure, external system configurations, and credentials must be declared within a root-level .env file. Below is the required .env.example structure to deliver to the client's infrastructure team:

```
# Redis Infrastructure Network Settings
REDIS_HOST=redis
REDIS_PORT=6379

# Telegram Monitoring & Notification Bot Alerts
BOT_TOKEN=1234567890:ABCdefGhIJKlmNoPQRsTUVwXyZ
ADMIN_ID=987654321

# Cryptographic Salt Configuration (ROTATE THIS IN PRODUCTION)
SECRET_SALT=YourProductionSecureCryptographicSaltString2026_ChangeMe

# System Logging and Archive Directories
SNAPSHOTS_DIR=/app/snapshots
LOG_LEVEL=INFO
```

5. Deployment Instructions via Docker Compose

The system is completely containerized and ready for cloud orchestration. It is highly recommended to deploy on an Ubuntu Linux instance (22.04/24.04 LTS) equipped with at least 4 CPU cores and 8 GB RAM to ensure uninterrupted handling of dense concurrent WebSocket streams.

Step-by-Step Deployment Steps:

1. Install Docker Engine and the modern Docker Compose plugin on the host system.
2. Clone the codebase into your designated application environment (e.g., /opt/quantum-terminal).
3. Instantiate the environment configuration file: `cp .env.example .env` and populate it with real API tokens and a secure cryptographic salt.
4. Build the core Docker images and spin up the microservices network in detached mode:

```
docker-compose up --build -d
```

5. Verify that all microservices and the infrastructure containers are running in a healthy state:

```
docker-compose ps
```

Monitoring System and Streaming Logs:

To monitor real-time websocket data ingestion or debug exchange connectivity problems, use Docker's logging mechanisms:

```
docker-compose logs -f ingester_binance
docker-compose logs -f api_server
```

6. Platform Administrator Operational Manual

Administrative provisioning and management tasks are done via quick interactive state updates inside the Redis instance.

Provisioning a New Commercial Client API Key:

To grant permanent commercial access to a client, execute the management script directly inside the running API server container. The script handles token generation (UUID4 with a TRADER- prefix) and updates the secure registry:

```
docker-compose exec api_server python api_server/key_manager.py "Client_Company_Name"
```

The script will print out the generated API key token which you can securely deliver to the end user.

Adding a New Pending Demo Key to the Registry:

If you wish to allow an evaluator to activate a 1-hour demo, add their pre-determined token string directly into the pending Set structure inside Redis:

```
docker-compose exec redis redis-cli SADD demo_keys_pending "DEMO-EVALUATIONTOKEN999"
```

Data Maintenance & Persistence Layout:

- **Redis Live Cache:** Redis is configured with persistence rules (`--appendonly yes`), ensuring that in-flight configurations and live key states survive system reboots inside the `redis_data` named volume.
- **Cold Analytical Snaps:** Parquet tables are saved in the `snapshots/` host-mapped directory. You should configure a scheduled cron or backup agent to mirror these files into an external storage bucket (e.g., AWS S3) once a week, as Redis automatically evicts historical minute records after 48 hours.